项目 12 智能物流-多功能综合实践

1、 整体课时安排

本项目位于实训课程的最后阶段，需要综合运用之前学到的视觉识别、运动控制、通信协议等技术，通过本项目可以综合考评对四足机器人的智能感知能力及综合运动性能的掌握情况。以下是本项目各子模块的课时安排。

项目名称	智能物流-多功能实践	学时
模块名称	项目需求及方案设计	2
	自动巡线模块	3
	自动卸货模块	3
	爬台阶模块	3
	避障模板	3
	项目测试及验收	2

2、 项目需求及系统设计

2.1、 项目需求

2.1.1、 应用场景

四足机器人作为移动机器人的重要分类，具备其他类型机器人无法比拟的优势，如更优秀的地形适应能力和稳定性，这使得它们在物资运送等任务中展现出巨大的潜力。

2.1.2、 场地标准

2.1.2.1、 场地规模与构造

地图以黄色盲道作为导航标识，宽度约 550mm，确保机器人能够准确追踪路径。

2.1.2.2、 材质选择与布置

选用 PU 布或喷绘布作为场地主要材质，为四足机器人提供了稳定的行走表面，保证了项目的顺畅进行。

2.1.2.3、 功能区域划分

场地内明确划分了多个功能区域，包含启停区、四个编号的倾倒区以及岔路口等，这些区域的设计考量了任务需求和场景多样性，为赛事增添了复杂性与挑战性。

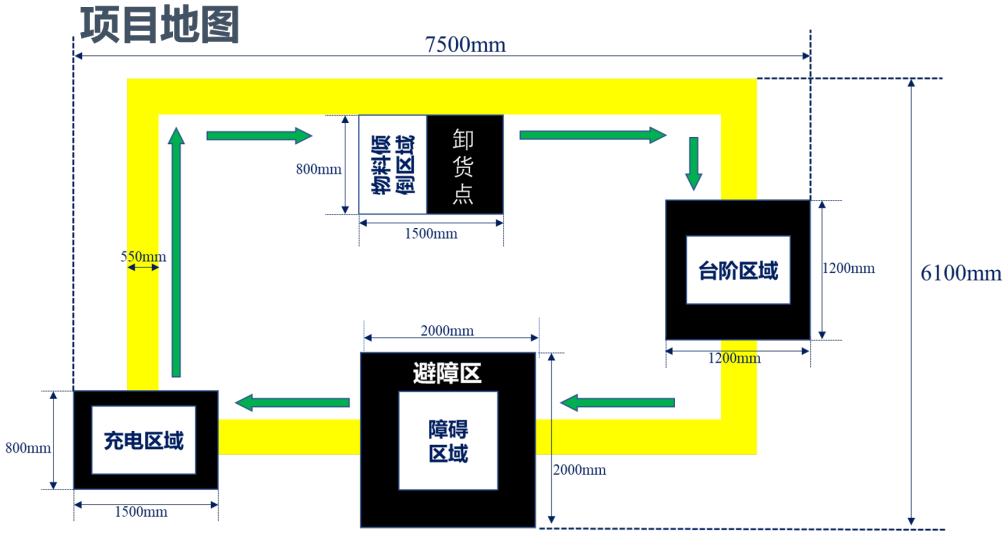
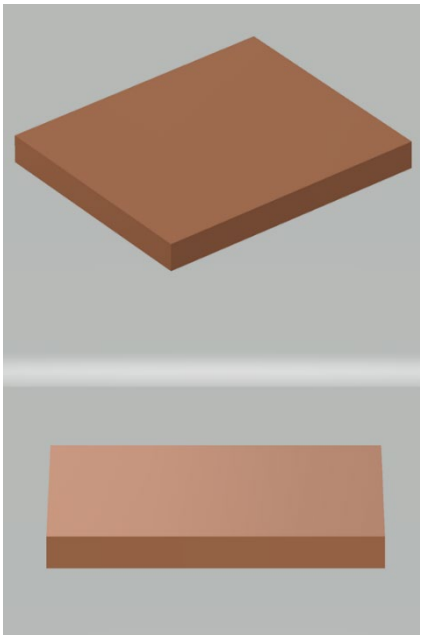


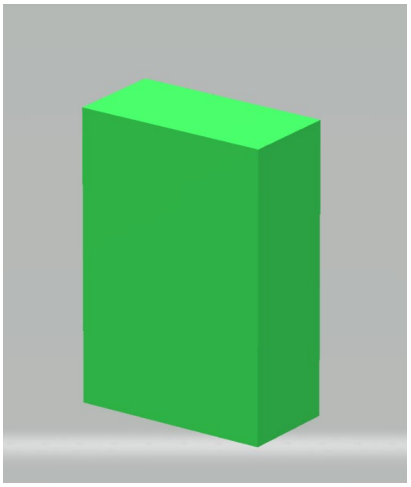
图 1-1 项目地图

2.1.2.4、 任务道具

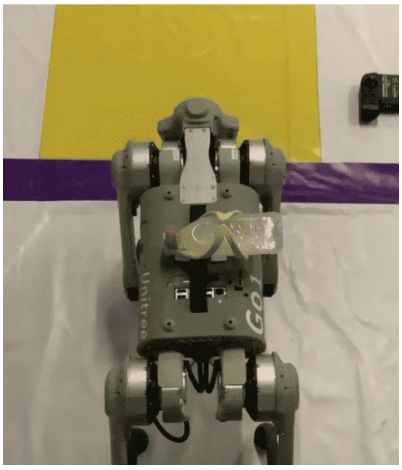
除了 Go2 机器人，我们在本项目中还需要以下任务道具。



台阶 规格：780*650*60mm



障碍物 规格：300*150*450mm



物料和背部载具

380ml 未开封瓶装水，自行设计一个载具来装载物料，材质、尺寸不限。

2.1.2.5、 任务说明

本项目模拟真实物资运送场景，通过四足机器人自带的视觉系统，从充电区域出发，获取周围环境图像信息，识别场景中的黄色道路，物料倾倒区域以及台阶等元素，到达指定终点（充电区域）。任务具体流程如下：

①携带物料从充电区域出发，沿黄色道路行走。

②识别到卸货点后，把物料卸载到物料倾倒区。

③完成卸载之后，继续沿着道路行走，通过台阶区域。

④到达障碍区域，通过规划在避障区完成避障。

⑤准确停在充电区域内，任务结束，统计完成时间。

2.2、 方案设计

2.2.1、 任务拆分

本系统任务拆分如下。

2.2.1.1、 环境识别

机器狗需要能够识别和适应比赛场景，包括不同的地面区域（如赛道、倾倒区、充电区等），以及各种障碍（如避障区障碍、台阶等）。

2.2.1.2、 路径规划

基于环境和任务的实时识别结果，进行动态路径规划，优化行进路线以避开障碍、减少完成时间。

2.2.1.3、 多样化步态

根据不同任务和地形设计多样化的步态，例如平地快速行走、跨越障碍物、上下楼梯的特殊步态。

2.2.1.4、 系统集成与测试

在模拟或实际环境中进行综合测试，确保机器狗能够适应所有任务和场景。

2.2.2、 任务实现

为实现以上任务，本系统设计为以下三个主要功能模块。

2.2.2.1、 视觉模块

识别黄色赛道（直行和转弯）。

识别任务区域（倾倒区域、台阶区域、障碍区域和充电区域）。

2.2.2.2、 决策模块

沿道路行走（直线行走或拐弯）。

到达任务区域执行特定动作。

2.2.2.3、 运动模块

实现基本步态：前进、左转、右转。

实现任务相关的动作组。

2.2.3、 软硬件架构

2.2.3.1、 硬件组成

四足机器人采用 GO2 机器狗，其硬件组成主要包括以下部分：

1、机身结构：包括机器人本体、四条机械腿和各个关节。

2、传感系统：包括 IMU（惯性测量单元）、激光雷达、视觉传感器（深度相机）。

3、控制系统：嵌入式计算机、微控制器，用于处理传感器数据和控制运动。

4、通讯模块：包括外接无线网卡或其他无线通讯模块，用于与外部设备（如控制电脑）进行通讯。

5、电源系统：包括电池和电源管理模块，为机器人提供动力。

2.2.3.2、 软件架构

软件系统整体采用两级架构：

1、视觉层：Python 程序用于读取外置深度相机信息，将视觉信

息进行处理后发送控制指令；

2、控制层：C++程序通过调用 SDK 底层接口控制四足机器人。接收到视觉程序的指令后，根据指令做出相应的任务状态更新和机器人控制。

3、UDP 通讯：两者之间采用 UDP 协议进行通讯。

注意：请根据各自系统的设计情况，也可采用 ROS2/DDS 通信协议。

2.2.4、 系统架构及流程设计

系统架构及具体流程说明如下。

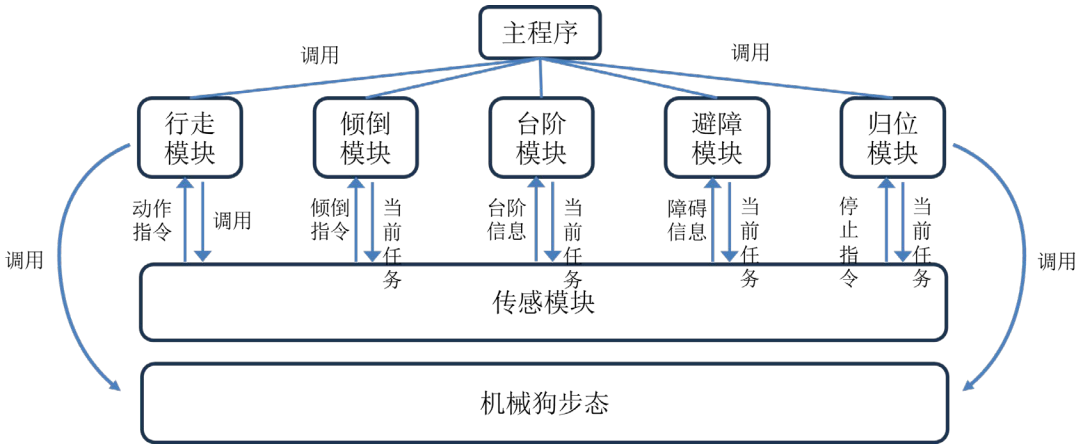


图 1-2 系统架构

2.2.4.1、 启动与初始化

①启动系统：启动机器人系统，确保所有硬件模块正常运行。

②初始化 UDP 通讯模块：创建 UDP 接收器对象并启动接收线程。设置 UDP 端口和 IP 地址，绑定端口用于接收视觉系统发送的指令。

③初始化运动控制模块：创建运动控制客户端对象并进行初始化，设置超时时间。初始化状态订阅器，订阅机器人状态信息，准备接收高频率的状态更新。

④等待启动信号：等待视觉系统发送启动信号，启动信号触发后开始任务执行流程。

2.2.4.2、 接收视觉信息

①UDP 接收线程运行：

UDP 接收线程不断监听来自视觉系统的 UDP 数据包，将接收到的数据存储在消息队列中。

②更新任务指令：

从消息队列中获取最新的指令信息（如“Turn Right”、“Unload”、“Climb”等），更新当前任务指令变量，以便在下一步决策时使用。

2.2.4.3、 任务决策与控制

①解析指令信息：

根据接收到的指令信息，判断当前任务状态。例如，如果收到“Turn Right”指令并且尚未完成该任务，则进入右转任务阶段。

②任务规划：

根据当前任务状态，规划机器人下一步行动。例如，在右转任务阶段，规划下一步的任务。

③运动控制：

向运动控制模块发送具体的运动指令。包括设定机器人运动的速度、方向和步态参数等，确保机器人按照规划路径和参数执行任务。

3、 自动巡线模块

3.1、 任务需求

本项目道路的标识颜色为黄色，需要正确识别道路与障碍线，障碍线的颜色包括紫色、红色、绿色和棕色。具体的参考颜色 RGB 值如下：黄色(255,255,0)、蓝色 RGB(0,0,255)、红色 RGB(255,0,0)、绿色 RGB(0,255,0)、棕色 RGB(165,42,42)、紫色 RGB(160,32,240)。

3.2、 实现思路及流程图

本项目中应用到的色阈通用检测方法如下。

①设定感兴趣的区域色 RGB 值的范围：根据颜色标准设定道路和障碍线的颜色范围。每种颜色都通过 RGB 值的上下限来定义。

②遍历像素点，筛选符合条件范围的像素点，形成掩膜：对视频帧或图像进行像素级遍历，筛选出处于设定颜色范围内的像素点，形

成颜色掩膜。

③掩膜后处理：对生成的颜色掩膜进行中值滤波和开运算操作，以消除噪声并平滑边界。

④边缘检测：应用边缘检测算法（如 Canny 边缘检测算法），对处理后的图像进行边缘检测，以获得道路和障碍线的精确边缘位置。

⑤输出道路和障碍线边缘：最终，从边缘检测结果中提取出道路和障碍线的边缘信息，用于后续的路径规划和导航决策。

本赛道采用明黄色材料制成，与周围环境及其他关键区域有明显的色差，采用色域判断法将简单高效的同时完成关键区域检测及方向判断两个任务。

首先将图像变换至 HSV 色彩空间上，然后调整色彩阈值，通过色彩阈值分割出明黄色赛道。设定目标区域，判断区域中是否有指定颜色的像素，并且遍历目标区域，统计区域内白色像素的数量。根据检测到的颜色区域，进行相应的控制决策：基于左上角和右下角区域的黄色像素数量，判断机器人的左右移动方向；基于左下角和右上角区域的黑色像素数量，判断机器人的上升或下降状态。

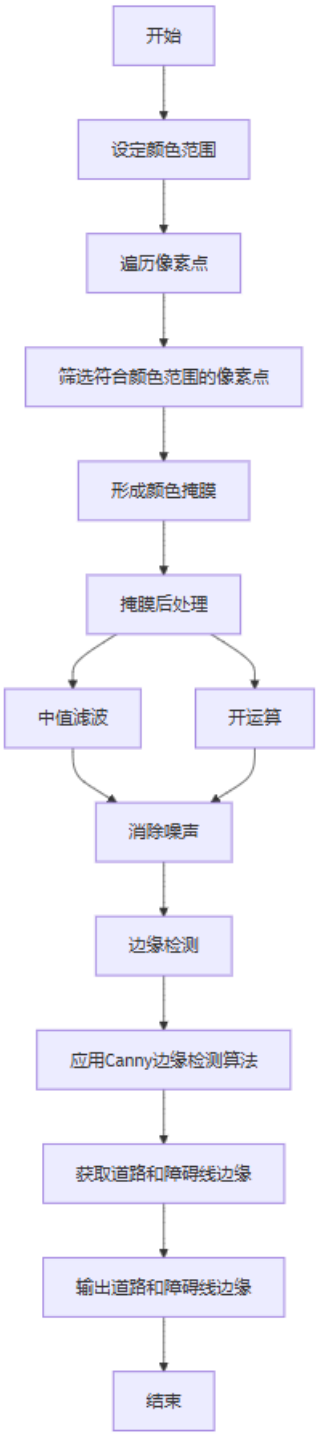

开始
设定颜色范围
遍历像素点
筛选符合颜色范围的像素点
形成颜色掩膜
掩膜后处理
中值滤波
开运算
消除噪声
边缘检测
应用Canny边缘检测算法
获取道路和障碍线边缘
输出道路和障碍线边缘
结束

图 3-1 色阈检测流程

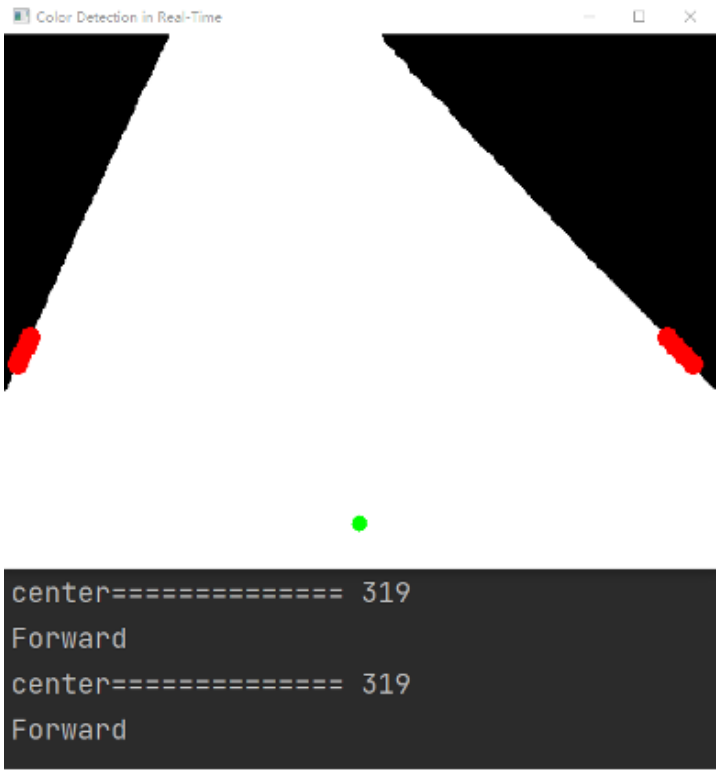


图 3-2 黄色跑道检测

3.3、 运动控制指令

根据当前跑道的实时检测情况，向机器人发出前进/偏左/偏右的运动指令。

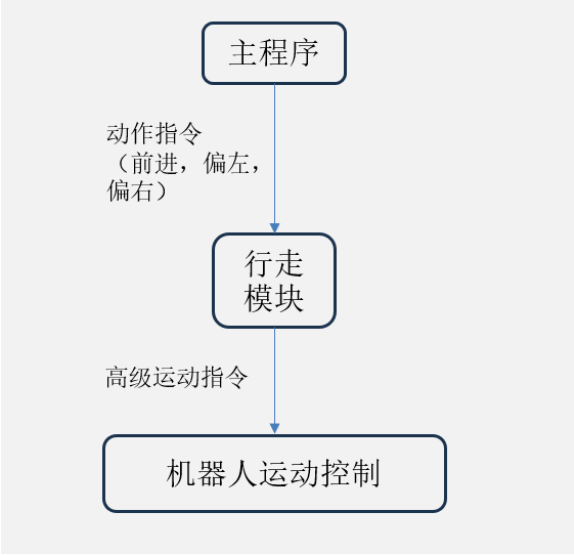


图 3-3 针对黄色跑道的运动控制

所用的高层运动函数如下。

函数名	Move
函数原型	int32_t Move(float vx, float vy, float vyaw)
功能概述	移动。
参数	**vx:** 取值范围[-2.5~3.8] (m/s)； **vy:** 取值范围[-1.0~1.0] (m/s)； **vyaw:** 取值范围[-4~4] (rad/s)。
返回值	调用成功返回 0，否则返回相关错误码。
备注	控制移动速度。设定的速度为机体坐标系表示下的速度。

函数名	StopMove
函数原型	int32_t StopMove()
功能概述	停下当前动作，将绝大多数指令恢复成默认值。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	停下当前的运动，并将 Go2 内部的运动参数恢复到默认值。

4、 自动卸货模块

4.1、 任务需求

识别到卸货点后，把物料卸载到物料倾倒区。

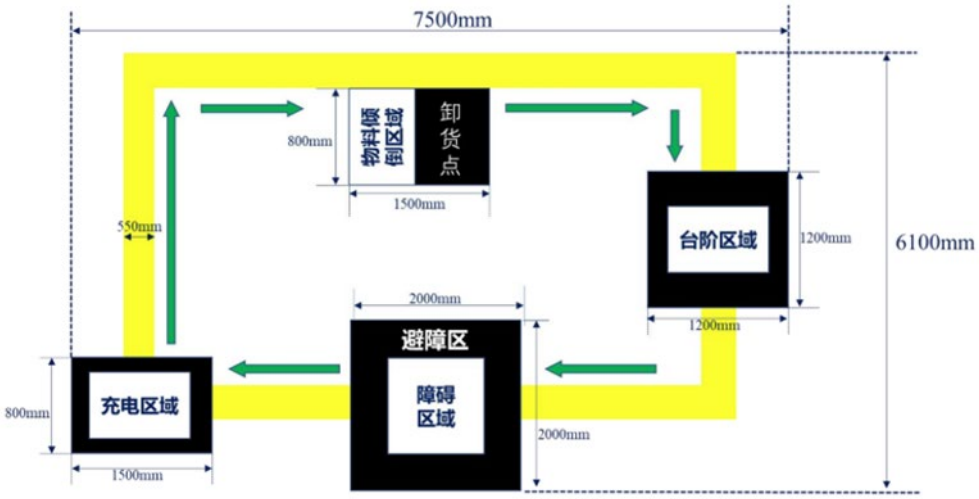


图 4-1 卸货点

4.2、 实现思路及流程图

4.2.1、 物料设计

自行设计物料框架用以固定水瓶，将物料架与四足狗装配连接。请注意倾斜角度，以保证前进途中物料不掉落。



图 4-2 物料固定图

4.2.2、 确认卸货点和物料倾倒区域

通过识别黑色，来判断是否进入卸货点。物料倾倒区与卸货点颜色差异较大，通过黑色色块面积的大小判断是否到达物料倾倒区并执行卸货任务。



图 4-3 卸货点与物料倾倒区域

具体采用基于 HSV 颜色空间的黑色区域检测算法。通过设定黑色的 HSV 阈值，并结合区域面积和位置验证，确保识别结果的准确性。此外，通过灰度分析进一步确认目标区域，确保机器人能够准确执行卸货任务。

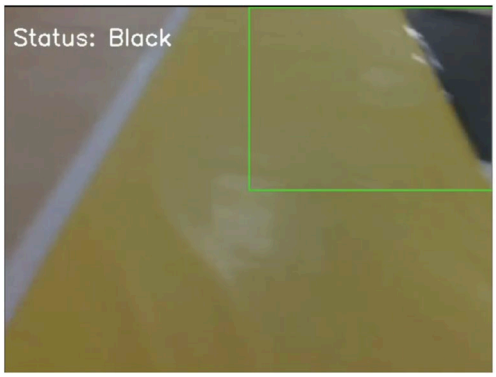
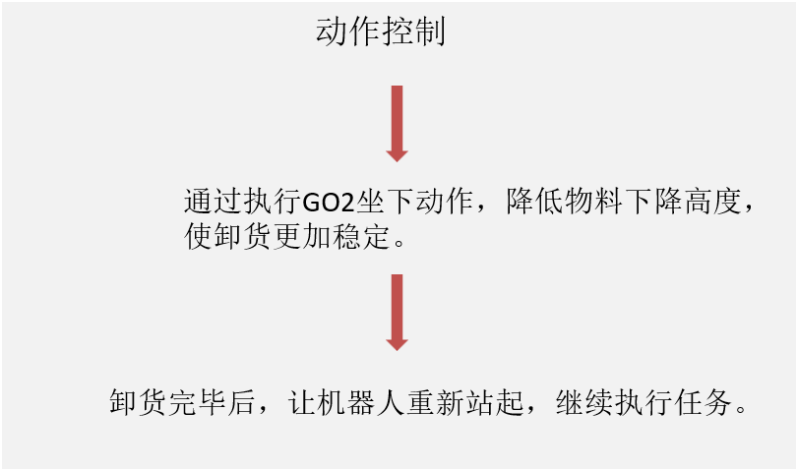


图 4-4 卸货区域实时识别

4.3、 运动控制指令

在本模块中给机器人的控制指令如下。



所用的高层运动函数如下。

函数名	Sit
函数原型	int32_t Sit();
功能概述	坐下。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	特殊动作，机器狗坐下。需要注意特殊动作需要在上一个动作执行完毕后再执行，否则会导致动作异常。

函数名	RiseSit
函数原型	int32_t RiseSit()
功能概述	站起（相对于坐下）。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	从坐下状态恢复到平衡站立

5、 爬台阶模块

5.1、 任务需求

完成物料卸载之后，继续沿着道路行走，通过台阶区域。

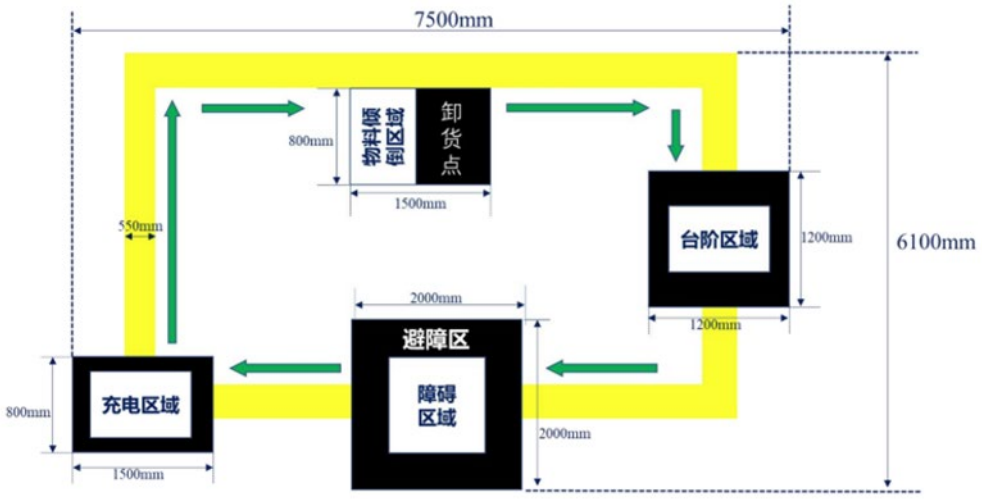


图 5-1 台阶区域

5.2、 实现思路及流程图

台阶区域也是黑色与周边颜色差异较大，这里与倾倒区的识别方

法相同，当满足预先设定的黑色区域面积，机器狗会开启爬楼模式。

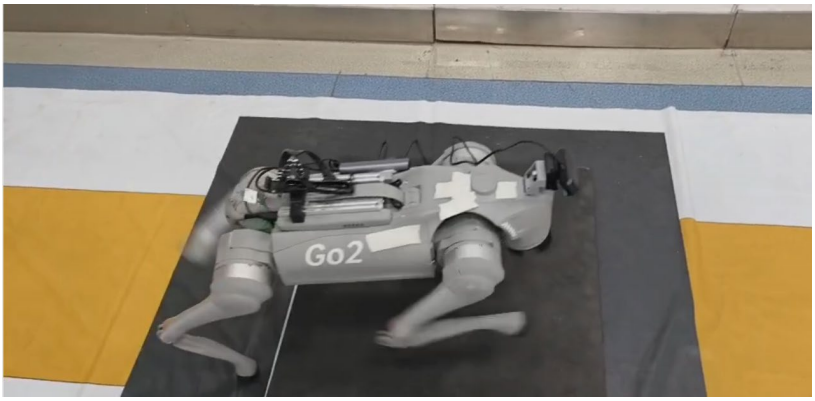


图 1-2 台阶区域

5.3、 运动控制指令

在本模块中给机器人的控制指令如下。

动作控制

↓

通过内置步态变换函数实现四足机器人上台阶功能。

↓

通过台阶后，让机器人恢复正常步态，继续执行任务。

所用的高层运动函数如下。

函数名	SwitchGait
函数原型	int32_t SwitchGait(int d)
功能概述	切换步态。
参数	d：步态枚举值，取值 0~4，0 为 idle，1 为 trot，2 为 trot running，3 正向爬楼模式，4：逆向爬楼模式。
返回值	调用成功返回 0，否则返回相关错误码。
备注	正向爬楼模式为机器人头朝楼梯，逆向爬楼模式为机器人背朝楼梯

6、 避障模块

6.1、 任务需求

通过台阶区域后紧接着的任务是障碍区域跨越，需要通过规划在避障区完成。赛道布置如下图所示，障碍区域中央放置了绿色障碍物作为示意，四足机器狗在此处需要绕行通过，再沿赛道走向终点，完成任务后停在指定的充电区域内。

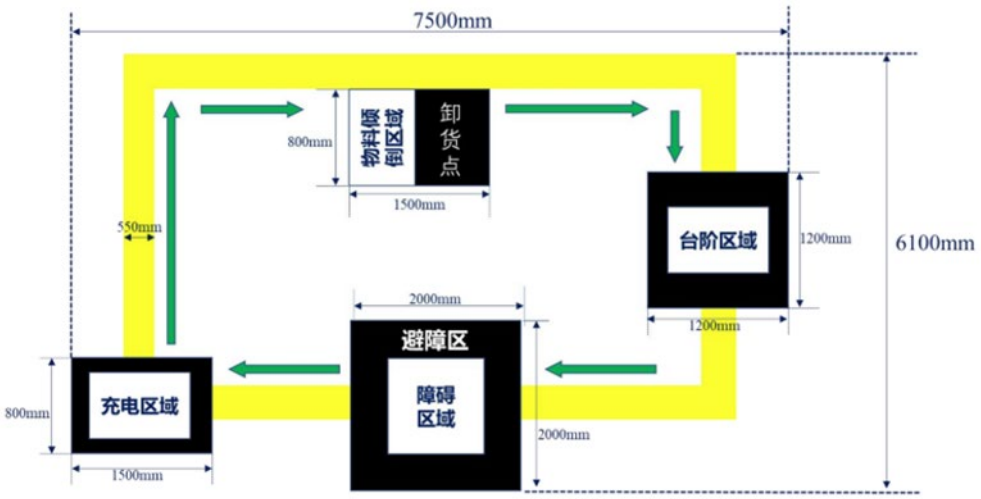


图 6-1 障碍区域

6.2、 实现思路及流程

6.2.1、 识别方案 1-摄像头视觉识别

采用基于 HSV 颜色空间的检测算法，通过设定本任务区几种颜色的 HSV 阈值，并结合区域面积和位置验证，确保识别结果的准确性。此外，通过灰度分析进一步确认落足区域，确保机器狗能够避开障碍，准确地行驶在有效区域内。

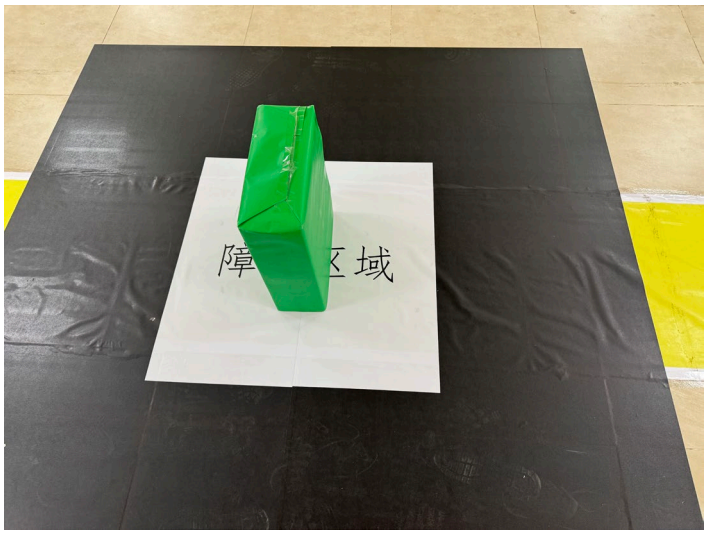


图 6-2 障碍区域

6.2.2、 识别方案 2-激光雷达障碍数据识别

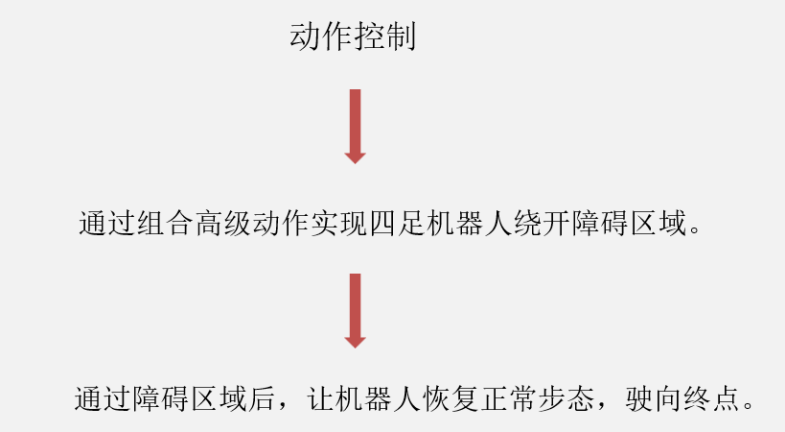
通过订阅 DDS 障碍物距离话题“rt/utlidar/range_info”，获取到激光雷达提供的障碍物距离信息，进行障碍物判断，然后执行避障方案。

```
$ ./subscribe_range_info
Received a range info message here!
  stamp = 1683716737.840576896
  frame = odom
  range front = 1.5
  range left = 0.7
  range right = 2.3
```

图 6-3 障碍物距离信息

6.3、 运动控制指令

通过组合动作实现避障任务。



7、 项目测试及验收

7.1、 项目指标

是否准确完成以下任务，统计总完成时间。

任务具体流程如下：

①携带物料从充电区域出发，沿黄色道路行走。

②识别到卸货点后，把物料卸载到物料倾倒区。

③完成卸载之后，继续沿着道路行走，通过台阶区域。

④到达障碍区域，通过规划在避障区完成避障。

⑤准确停在充电区域内，任务结束。

7.2、 项目测试及优化

7.2.1、 模拟物流场景，制定测试方案

模拟物流场景，包括黄色道路、物料倾倒区域、台阶、避障区域和充电区域等。多次运行控制程序，记录机器人在各个任务阶段的表

现。

7.2.2、 记录测试结果，优化解决方案

记录每次的测试结果，测试机器人能否顺利沿着黄色道路行走，成功完成卸货、爬坡、避障和到达充电区域的任务。

如果机器人会运动偏移出赛道范围，运动控制不够平稳等。针对测试中发现的问题，完善控制算法和识别算法，提高机器人的运动稳定性和任务执行效率。

7.2.3、 项目验收及项目总结

系统测试完成后找任课教师进行项目验收，然后进行项目总结，最后撰写并提交项目报告。